Rapport de Projet : IMAG Dungeon

Architecture logicielle, conception orientée objet et validation

Labib Begdouri Terraf, Saif-Eddine Mehrach, Reda Alou, Moad EL Sofi

Octobre 2026

1 Présentation et Objectifs

Le projet **IMAG Dungeon** est un jeu d'exploration et de combat au tour par tour en console. L'architecture respecte les principes de la programmation orientée objet (POO) pour modéliser une maison hostile (salles, portes verrouillées, monstres, trésor et sortie). Le cycle de développement s'appuie sur un `Makefile` standardisé (`make`, `make run`, `make clean`) avec un point d'entrée unique `EntryPoint.java` déléguant à `MenuConfig`.

Périmètre fonctionnel réalisé :

— Parsing déclaratif complet de la carte depuis un fichier texte avec vérification d'intégrité (`LecteurMaison`).

— Gestion dynamique du graphe spatial (salles, portes à clés, coordonnées cartésiennes discrètes).

— Moteur d'inventaire contraint par capacité et encombrement (`Sac`).

— Moteur de combat en tour par tour asymétrique avec gestion des alliés, trahisons et inventaire actif.

— Radar de proximité et rendu textuel de la carte (`Affichage`).

2 Choix de Conception

2.1 Architecture en Modules Découplés

Le code est organisé en cinq packages selon une hiérarchie stricte de dépendances unidirectionnelles :

`Position` ⟶ `SAC` ⟶ `Personnages` ⟶ `MAPS` ⟶ `Menu`

— **`Position`** : Primitives spatiales géométriques immuables (`Position`, `Direction`).

— **`SAC`** : Système d'inventaire et objets manipulables (`Item`, `Arme`, `Cle`, `Food`, `Tresor`, `Sac`).

— **`Personnages`** : Entités vivantes autonomes ou incarnées, boucle de duel (`Personnage`, `Heros`, `Monstre`, `Allie`, `Combat`).

— **`MAPS`** : Topologie du donjon, parsing et rendu (`Maison`, `Salle`, `Porte`, `LecteurMaison`, `Affichage`).

— **`Menu`** : Orchestration de la boucle de jeu et traitement des entrées utilisateur (`MenuConfig`).

Justification : Cette stratification empêche tout couplage cyclique. Les entités de données de bas niveau (`Item`, `Position`) ignorent totalement la logique applicative et l'interface utilisateur.

2.2 Héritage et Polymorphisme

Deux hiérarchies majeures structurent le domaine :

— **Hiérarchie `Personnage` (Abstraite) :** Spécialisée en `Heros`, `Monstre` et `Allie`.

  — *Facteurs communs :* `nom`, `pv`, `pvMax`, `position`, ainsi que les primitives d'état (`getEstMort()`, `subirDegats()`).

— **Hiérarchie `Item` (Abstraite) :** Spécialisée en `Arme` (dégâts), `Cle` (identifiant de serrure), `Food` (soin instantané) et `Tresor` (condition de victoire).

— *Facteurs communs :* `nom`, `encombrement`, `position`.

— *Polymorphisme :* Chaque sous-type surcharge sa méthode de description/représentation radar et son comportement d'utilisation dans le sac.

2.3 Persistance et Format de Carte

La carte est chargée via un format déclaratif structuré par ligne (`LecteurMaison`) :

— `Salle:<nom>:<h>:<l>:<x0>:<y0>` : Définit l'enveloppe rectangulaire et l'ancrage spatial de la pièce.

— `PORTE:<O|F>:<idCle>:<salleDest>:<x>:<y>` : Encapsule la transition entre deux salles et sa condition d'accès.

— `ARME`, `ALLIE`, `FOOD`, `TRESOR`, `REVEIL` : Instancient et positionnent les entités de départ.

Robustesse et encapsulation : Tous les attributs sont privés. Les méthodes de mutation vérifient systématiquement les invariants (PV non négatifs, encombrement n'excédant pas le volume max du sac, coordonnées strictement positives).

3 Utilisation des Classes et Justification des Méthodes

Classe	Méthodes Clés	Rôle & Justification POO
`Personnage`	`attaquer()`, `subirDegats()`	Classe abstraite garantissant un contrat uniforme de vie et de dégâts.
`Heros`	`consommer()`, `ramasser()`	Spécialisation dotée d'un sac et de la liste des alliés recrutés.
`Monstre`	`getArme()`	Entité adverse armée réagissant aux combats de salle.
`Allie`	`getEstTraitre()`	Entité pouvant basculer dynamiquement de camp lors des rencontres.
`Combat`	`Combattre()`, `lireEntier()`	Moteur tour par tour découplé ; gère la finitude et la validité des entrées.
`MenuConfig`	`boucleJeu()`, `gererDeplacement()`	Contrôleur de flux orienté événement reliant commandes et état.
`Sac`	`ajouter()`, `retirer()`, `estPlein()`	Encapsulation de la règle métier d'encombrement fini.
`Porte`	`deverrouiller(Cle)`	Vérification déterministe entre l'identifiant de la porte et de la clé.
`Position`	`deplacer(Direction)`, `distance()`	Objet-valeur immuable modélisant l'espace 2D.
`LecteurMaison`	`chargerMaison(String)`	Fabrique logicielle convertissant le fichier texte en graphe de mémoire.

4 Stratégie de Combat & Déroulement d'un Tour

Le système implémente un affrontement en tour par tour géré par `Combat.java` opposant l'équipe du joueur (`Heros`, `Allie`) au camp hostile (`Monstre`, `AllieTraitre`).

4.1 Séquencement Temporel

1. **Phase Héros (Décisionnelle) :** Le joueur choisit entre *Attaquer* (focus ciblé via `choixMonstre()`), *Changer d'arme* (action gratuite ne consommant pas le tour), ou *Consommer de la nourriture* (`Food`) pour régénérer ses points de vie.
2. **Phase Alliés (Autonome) :** Les alliés vivants attaquent un ennemi choisi aléatoirement via `Random.nextInt()`.

3. **Phase Ennemis (Adverse) :** Les adversaires attaquent un membre du groupe joueur au hasard. La confrontation s'interrompt immédiatement en cas de mort du héros ou d'extermination ennemie.

4.2 Limites Identifiées du Moteur de Combat

— **IA élémentaire :** Ciblage stochastique sans prise en compte des PV faibles. *Piste :* Patron **Strategy**.

— **Couplage I/O :** Présence directe de `System.out` et `Scanner`. *Piste :* Patron **MVC**.

5 Tests Effectués et Résultats

Les validations ont été conduites sur la carte d'intégration `MaisonTest.txt` :

— **Déplacement et radar :** Déplacement vers un mur puis vers une zone libre. *Attendu :* Blocage sur obstacle et mise à jour des coordonnées. *Résultat :* Conforme, pas de traversée illicite de murs.

— **Portes et serrures :** Tentative d'ouverture d'une porte fermée avec mauvaise clé, puis bonne clé. *Attendu :* Rejet puis déverrouillage persistant. *Résultat :* Conforme, la porte reste ouverte.

— **Contrainte du Sac :** Ramassage d'un item dépassant le volume limite. *Attendu :* Refus d'ajout avec notification console sans perte d'objet au sol. *Résultat :* Conforme.

— **Consommation de soin :** Consommation de `Food` en sous-santé. *Attendu :* Augmentation des PV sans dépasser `pvMax`. *Résultat :* Conforme.

— **Combat et états de fin :** Simulation d'une victoire (ennemis vaincus) et défaite (héros à 0 PV). *Attendu :* Nettoyage des morts et arrêt immédiat de la boucle. *Résultat :* Conforme.

— **Alliés et trahisons :** Rencontre d'un allié loyal vs traître. *Attendu :* L'allié loyal rejoint `avec`, le traître intègre `contre`. *Résultat :* Conforme.

— **Condition de victoire :** Atteinte de la salle de sortie sans trésor, puis avec le trésor. *Attendu :* Blocage de fin de partie sans le trésor ; message de victoire sinon. *Résultat :* Conforme.

— **Robustesse I/O :** Saisie de caractères textuels dans les menus numériques (`lireEntier`). *Attendu :* Absence de crash (`InputMismatchException`) et re-prompt. *Résultat :* Conforme.

6 Conclusion

Ce projet a permis de concrétiser les patrons majeurs de la POO (héritage, encapsulation stricte, polymorphisme et séparation en packages). La principale difficulté a résidé dans la synchronisation spatiale entre le graphe des salles et l'affichage radar.